Southern New Hampshire University

Andres Trujillo
CS 410 Professor Parker
C++ to Assembly with loops Activity
May 12, 2026

C++ Code Functionality

C++ Line of Code	Explanation of Functionality
#include <iostream>	This includes a library for input and output streams that we will use to print to our local terminal window buffer.
using namespace std;	A compile-time directive that tells the compiler to search the 'std' namespace without writing "std::"
int main() { .. }	The entry point of our program from the C++ runtime library after global and static constructors are initialized by the C/C++ runtime library on the host machine.
int num, i, product = 1;	These are primitive types of type integer where product is being assigned to the number one and the others are left uninitialized.
cout << "Enter a number:\n";	This line outputs to the terminal buffer window a string.
cin >> num;	Gets user input and saves integer to num variable.
for (i = num; i > 0; --i){	This initializes a for-loop then sets our previously unused variable 'i' to the input number and it then separates the initialization and conditional to check whether the newly initialized value is greater than zero. And then after the final curly brace is reached from the initial curly brace of this line we decrement the value by one and the for-loop runs again after checking the condition described earlier.
product = product * i;	This multiplies the number one by a number, then subtracts one from a number and multiplies it to the result of the previous iteration until the number is equal to zero where the loop will end.
cout << "The factorial for " << num << "is: " << product << endl;	This line outputs to the terminal buffer window.
return 0;	Returns the value zero to the CRT and exits the function.

C++ to Assembly Alignment

C++ Line of Code	Blocks of Assembly Code
#include <iostream>	.globl _ZSt21ios_base_library_initv
using namespace std;	.. part of above assembly code
int main() { .. }	main:
int num, i, product = 1;	movl $1, -8(%rbp)
cout << "Enter a number:\n";	call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@PLT
cin >> num;	leaq _ZSt3cin(%rip), %rax movq %rax, %rdi call _ZNSirsERi@PLT
for (i = num; i > 0; --i){	movl -12(%rbp), %eax movl %eax, -4(%rbp) cmpl $0, -4(%rbp) jg .L3 subl $1, -4(%rbp)
product = product * i;	movl -8(%rbp), %eax imull -4(%rbp), %eax movl %eax, -8(%rbp)
cout >> "The factorial for " >> num >> "is: " >> product >> endl;	leaq .LC1(%rip), %rax movq %rax, %rsi leaq _ZSt4cout(%rip), %rax movq %rax, %rdi call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@PLT movq %rax, %rdx movl -12(%rbp), %eax movl %eax, %esi movq %rdx, %rdi call _ZNSolsEi@PLT movq %rax, %rdx leaq .LC2(%rip), %rax movq %rax, %rsi movq %rdx, %rdi call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@PLT movq %rax, %rdx movl -8(%rbp), %eax movl %eax, %esi movq %rdx, %rdi call _ZNSolsEi@PLT movq _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@GOTPCREL(%rip), %rdx movq %rdx, %rsi movq %rax, %rdi call _ZNSolsEPFRSoS_E@PLT
return 0;	movl $0, %eax

	leave ret

Assembly Functionality

Blocks of Assembly Code	Explanation of Functionality
.file "AssemblyWithLoops.cpp"	Declares the name of the source file.
.text	Defines text portion of assembly file containing machine instructions, and functions.
.globl _ZSt21ios_base_library_initv	Imports the input and output streaming library.
.section .rodata	Places data into read only area of stack. The compiler is creating constants and writing them to their location for the process to run effectively
.LC0: .string "Enter a number:\n"	Saves a string from source file.
.LC1: .string "The factorial for "	... same
.LC2: .string "is: "	... same
.text .globl main .type main, @function	This .text line switches the compiler back into executable code section after switching to read only section and exports symbol main for the linker and that main is a function symbol not an object or other data types.
main:	Entry point for function
movl $1, -8(%rbp)	Defines the product variable and sets it to one.
call _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@PLT	Prints to standard output our text.
leaq _ZSt3cin(%rip), %rax movq %rax, %rdi call _ZNSirsERi@PLT	Gets input from user.
movl -12(%rbp), %eax movl %eax, -4(%rbp) cmpl $0, -4(%rbp) jg .L3 subl $1, -4(%rbp)	Initializes for loop and jump points to executing code.
movl -8(%rbp), %eax imull -4(%rbp), %eax movl %eax, -8(%rbp)	This part is the loop body that figures our factorial by multiplying from the start of a number and subtracting one from it and multiplying it by its previous value.
leaq .LC1(%rip), %rax movq %rax, %rsi leaq _ZSt4cout(%rip), %rax movq %rax, %rdi call _ZStlsISt11char_traitsIcEERSt13basic_os	This part prints the rest of our string with the value after computation. It breaks up the standard out calls to add the strings and integers into the stream buffer so that it can be displayed on our screen.

treamIcT_ES5_PKc@PLT movq %rax, %rdx movl -12(%rbp), %eax movl %eax, %esi movq %rdx, %rdi call _ZNSolsEi@PLT movq %rax, %rdx leaq .LC2(%rip), %rax movq %rax, %rsi movq %rdx, %rdi call _ZStlsISt11char_traitsIcEERSt13basic_os treamIcT_ES5_PKc@PLT movq %rax, %rdx movl -8(%rbp), %eax movl %eax, %esi movq %rdx, %rdi call _ZNSolsEi@PLT movq _ZSt4endlIcSt11char_traitsIcEERSt13bas ic_ostreamIT_T0_ES6_@GOTPCREL(%rip), %rdx movq %rdx, %rsi movq %rax, %rdi call _ZNSolsEPFRSoS_E@PLT	
movl $0, %eax leave ret	This returns the stack pointer to the start of the function and sets the return value to zero and exits the function.